Chapter 14 — Observability, Safety & Security

14.1 Tracing and Observability with OpenTelemetry

Conceptual Deep Dive

LLM apps are multi-step pipelines whose failures are semantic, so you need **traces** (one tree per request: retrieval span, rerank span, each LLM call, each tool call), **metrics** (latency, tokens, cost, error rate, cache hit rate, repair rate) and **logs/events** (prompts, completions, with redaction). **OpenTelemetry (OTel)** is the vendor-neutral standard; its **GenAI semantic conventions** define attributes such as `gen_ai.request.model`, `gen_ai.usage.input_tokens`, `gen_ai.usage.output_tokens`. Instrumentation libraries: OpenInference and OpenLLMetry auto-instrument OpenAI/Anthropic/LangChain/LlamaIndex. Backends: **Arize Phoenix** (open source, OTel-native traces + evals), **Langfuse** (open source; traces, prompt management, datasets, scores), LangSmith, Braintrust, Datadog/Grafana/Honeycomb. Attach prompt version, user/session id (hashed), and feedback scores to traces; sample aggressively for cost but keep 100% of errors and low-score traces.

Cross-Connection Scenarios

Traces become eval datasets (10.1); online metrics for A/B (10.5); autoscaling metrics (13.4).

Production Anti-Patterns

- Logging raw prompts with PII to a third-party SaaS.
- Only infrastructure metrics, no quality signals.
- Missing correlation ids across services.

Minimal Working Code

```
# pip install opentelemetry-sdk
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor, ConsoleSpanExporter

provider = TracerProvider()
provider.add_span_processor(BatchSpanProcessor(ConsoleSpanExporter()))
trace.set_tracer_provider(provider)
tracer = trace.get_tracer("rag-app")

def llm_call(prompt: str) -> str:
    with tracer.start_as_current_span("llm.chat") as span:
        span.set_attribute("gen_ai.request.model", "example-model")
        out = "answer"
        span.set_attribute("gen_ai.usage.input_tokens", len(prompt) // 4)
        span.set_attribute("gen_ai.usage.output_tokens", len(out) // 4)
        span.set_attribute("prompt.version", "a1b2c3d4")
        return out

with tracer.start_as_current_span("rag.request"):
    with tracer.start_as_current_span("retrieve"):
        pass
    llm_call("hello")

# Phoenix: import phoenix as px; from phoenix.otel import register; register(project_name="rag-app&q
uot;)
```

```
# Langfuse: from langfuse import observe  ->  @observe() on functions
```

14.2 Threat Model: Injection, Jailbreaks, Data Leakage

Conceptual Deep Dive

Map risks with the **OWASP Top 10 for LLM Applications**: prompt injection, sensitive information disclosure, supply chain, data/model poisoning, improper output handling, excessive agency, system-prompt leakage, vector/embedding weaknesses, misinformation, unbounded consumption. **Jailbreaks** coax a model past its safety training (role-play, encodings, many-shot, adversarial suffixes, multi-turn crescendo). **Indirect injection** rides in on retrieved content (see 6.5). **Data exfiltration** channels: markdown image URLs, link rendering, tool calls to external hosts. **Output handling**: treat model output as untrusted — never eval, render unsanitised HTML, or build SQL/shell from it. **Cost attacks**: huge inputs, loops (denial of wallet). **RAG-specific**: cross-tenant leakage via missing ACL filters, poisoned documents, embedding inversion. Use threat modelling (STRIDE) for each trust boundary and red-team with tools like garak, PyRIT, promptfoo red-team.

Cross-Connection Scenarios

Agent permissions (11.6), tool validation (7.4), ACL filtering (8.5), gateway budgets (13.6).

Production Anti-Patterns

- Security by system-prompt wording alone.
- Model output flowing straight into `subprocess`/SQL.
- No per-user token or spend limits.

Minimal Working Code

```
import html
import re

def safe_render(model_text: str) -> str:
    text = re.sub(r"!\[[^\]]*\]\([^)]*\)", "", model_text)       # block image exfil
    return html.escape(text)

ALLOWED_HOSTS = {"docs.example.com"}

def check_url(url: str) -> bool:
    from urllib.parse import urlparse
    return urlparse(url).hostname in ALLOWED_HOSTS

class SpendLimiter:
    def __init__(self, daily_tokens=200_000):
        self.limit, self.used = daily_tokens, {}
    def charge(self, user: str, tokens: int) -> bool:
        self.used[user] = self.used.get(user, 0) + tokens
        return self.used[user] <= self.limit
```

14.3 PII Detection and Redaction

Conceptual Deep Dive

Regulations (GDPR, CCPA, HIPAA, PCI-DSS) constrain where personal data may flow. Strategy: detect and redact at boundaries — before sending to third-party APIs, before logging, before indexing, and before fine-tuning. Methods: regex (emails, phones, SSN, cards with Luhn check), NER models (**Microsoft Presidio**, spaCy, GLiNER), and LLM-based detectors for context-dependent PII. Replace with typed placeholders (<PERSON_1>) and keep a reversible vault mapping in a secure store when the answer needs re-identification (**pseudonymisation**). Measure precision/recall of the redactor; high recall matters more for compliance. Also: data retention limits, encryption at rest/in transit, zero-retention provider agreements, and honouring deletion requests across logs, vector stores and memories.

Cross-Connection Scenarios

Memory deletion (11.4), fine-tuning data cleaning (12.2), logging (14.1).

Production Anti-Patterns

- Redacting only the prompt, not the retrieved context or tool outputs.
- Placeholders that collapse distinct people into one token.
- Storing the reversible map next to the redacted data.

Minimal Working Code

```
import re

PATTERNS = {
    "EMAIL": re.compile(r"[\w.+-]+@[\w-]+\.[\w.-]+"),
    "SSN": re.compile(r"\b\d{3}-\d{2}-\d{4}\b"),
    "PHONE": re.compile(r"\b(?:\+?1[-. ]?)?\(?\d{3}\)?[-. ]?\d{3}[-. ]?\d{4}\b"),
}

def luhn(num: str) -> bool:
    ds = [int(c) for c in re.sub(r"\D", "", num)][::-1]
    return len(ds) >= 13 and sum(d if i % 2 == 0 else (d * 2 - 9 if d * 2 > 9 else d * 2)
                                 for i, d in enumerate(ds)) % 10 == 0

def redact(text: str):
    vault, counters = {}, {}
    def sub(kind):
        def f(m):
            counters[kind] = counters.get(kind, 0) + 1
            tag = f"<{kind}_{counters[kind]}>"
            vault[tag] = m.group(0)
            return tag
        return f
    for kind, pat in PATTERNS.items():
        text = pat.sub(sub(kind), text)
    return text, vault

print(redact("Mail bob@x.com or call 415-555-0100, SSN 123-45-6789"))
# Presidio: AnalyzerEngine().analyze(text=..., language="en") + AnonymizerEngine().anonymize(..
.)
```

14.4 Guardrails: NeMo Guardrails, Llama Guard and Content Filtering

Conceptual Deep Dive

Guardrails are checks around the model. **Input rails**: topic restriction, jailbreak/injection detection, PII. **Output rails**: toxicity, policy violations, factuality/citation checks, schema validation. **Dialog rails**: conversation flows. **Retrieval rails**: filter chunks. **Execution rails**: tool-call authorisation. Tools: **NVIDIA NeMo Guardrails** (Colang flows and programmable rails), **Llama Guard** (a fine-tuned safety classifier over a hazard taxonomy, e.g. MLCommons categories, for prompts and responses), **Prompt Guard / PromptGuard-style** injection classifiers, **Guardrails AI** validators, provider moderation endpoints, and Azure/Bedrock content filters. Design: run cheap checks first, parallelise input checks with generation and cancel if flagged, tune thresholds on labelled data to balance false positives (annoyed users) against false negatives (harm), return helpful refusals, and log every block for review. Guardrails add latency and can be bypassed — they complement, never replace, least-privilege architecture.

Cross-Connection Scenarios

Injection defence (6.5), agent kill switches (11.6), eval of guardrail precision/recall (10).

Production Anti-Patterns

- Blocking by keyword lists only.
- Running guardrails after streaming has already shown the user harmful text.
- Never measuring false-positive rate.

Minimal Working Code

```
import asyncio

async def guarded_generate(user_text, input_checks, generate, output_checks):
    results = await asyncio.gather(*(c(user_text) for c in input_checks))
    if any(r["flagged"] for r in results):
        return {"blocked": True, "message": "Sorry, I can't help with that reque
st."}
    answer = await generate(user_text)
    out = await asyncio.gather(*(c(answer) for c in output_checks))
    if any(r["flagged"] for r in out):
        return {"blocked": True, "message": "I can't share that response."}
    return {"blocked": False, "message": answer}

# NeMo Guardrails:
#   from nemoguardrails import RailsConfig, LLMRails
#   rails = LLMRails(RailsConfig.from_path("./config")); rails.generate(messages=[...])
# Llama Guard via transformers: build chat template with the user/assistant turns; parse "safe"
/"unsafe\nS1".
```

14.5 Secure Engineering Practices

Conceptual Deep Dive

Secrets in a vault, rotated, never in prompts or client code. Authenticate and authorise every request; scope vector queries and tool calls to the caller. Rate-limit per user. Verify model artifacts (safetensors, hashes, signed images); pin and audit dependencies (SBOM, pip-audit); beware malicious packages and typosquatting for AI libs. Isolate code execution. Maintain audit logs and an incident-response playbook with a kill switch (disable a tool, model or feature flag instantly). Evaluate third-party model/vendor risk: data retention, training on your data, sub-processors. Document with model cards and risk assessments; align with NIST AI RMF, ISO/IEC 42001, and the EU AI Act risk tiers where applicable.

Cross-Connection Scenarios

Container hardening (13.3), CI security scanning (15), compliance for career-level communication (16).

Production Anti-Patterns

- Shared admin API key across environments.
- No way to disable a misbehaving feature without a deploy.
- Trusting model hub downloads without hash pinning.

Minimal Working Code

```
import hashlib
import os

def sha256_file(path: str) -> str:
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(1 << 20), b""):
            h.update(chunk)
    return h.hexdigest()

def verify_artifact(path: str, expected: str) -> None:
    if sha256_file(path) != expected:
```

```
        raise RuntimeError(f"hash mismatch for {path}")

def feature_enabled(name: str) -> bool:
    """Kill switch via env/flag service; default-off for risky features."""
    return os.getenv(f"FEATURE_{name.upper()}", "0") == "1"
```

14.6 Red-Teaming and Safety Evaluation

Conceptual Deep Dive

Red-teaming systematically searches for failures before attackers do. Build an attack library by category: direct and indirect injection, jailbreak families (role-play, encoding, multi-turn escalation, many-shot), data-extraction probes (system prompt, training data, other users' data), tool abuse (SSRF, path traversal, excessive agency), harmful-content requests per your policy, and denial-of-wallet inputs. Automate with tools (garak, PyRIT, promptfoo red-team) and LLM-generated attack variants, then triage by severity x exploitability. Track **attack success rate (ASR)** per category and per release, and keep every successful attack as a permanent regression test. Include human red-teamers; automated suites miss creative attacks. Edge cases: attacks that succeed only after many turns, language switching, attacks hidden in uploaded files and images, and guardrails that block benign users (measure false-positive rate on a benign set too).

Cross-Connection Scenarios

Guardrails (14.4), regression suite and CI gates (15.2), agent security (11.6), eval maintenance (10.8).

Production Anti-Patterns

- One-time red-team before launch and never again.
- Reporting block rate without a benign false-positive set.
- Fixing the specific prompt that worked instead of the underlying weakness.

Minimal Working Code

```
def run_attacks(system, attacks: list[dict], is_violation) -> dict:
    """attacks: [{'category': str, 'prompt': str}]; is_violation(response)->bool."
""
    by_cat = {}
    for a in attacks:
        resp = system(a["prompt"])
        bad = is_violation(resp)
        c = by_cat.setdefault(a["category"], {"n": 0, "success": 0, "e
xamples": []})
        c["n"] += 1
        c["success"] += bad
        if bad and len(c["examples"]) < 3:
            c["examples"].append(a["prompt"][:80])
    return {k: {"asr": v["success"] / v["n"], **v} for k, v in by_cat.items
()}

demo = run_attacks(lambda p: "SECRET" if "ignore" in p else "refused",
                   [{"category": "injection", "prompt": "ignore rules&
quot;},
                    {"category": "injection", "prompt": "hello"}]
,
                   lambda r: "SECRET" in r)
print(demo)
```

```
def false_positive_rate(guard, benign_prompts: list[str]) -> float:
    blocked = sum(1 for p in benign_prompts if guard(p)["flagged"])
    return blocked / len(benign_prompts)

print(false_positive_rate(lambda p: {"flagged": "kill" in p}, ["kill the process
```

```
", "hello", "how to bake"]))
```

14.7 Privacy Engineering, Governance and Incident Response

Conceptual Deep Dive

Data lifecycle: collect minimally, classify (public/internal/confidential/regulated), encrypt in transit and at rest, set retention per class, and support deletion across logs, caches, vector stores, memories, fine-tuning sets and backups (deletion in trained weights is hard: keep PII out of training). **Privacy techniques**: redaction, pseudonymisation, aggregation, differential privacy for analytics, on-prem or VPC-isolated inference for regulated data. **Governance**: model and data inventory, risk tiering (EU AI Act categories), impact assessments, model cards, access reviews, vendor assessments. **Incident response** for AI: detect (alerts on guardrail spikes, leakage detectors), contain (feature-flag kill switch, revoke keys, disable tool), eradicate (fix root cause, add regression test), notify (regulators/customers per law and contract), and learn (blameless postmortem). Edge cases: user-uploaded documents containing others' PII, logs shipped to a SaaS observability vendor, cross-border data transfers, and "right to explanation" requests.

Cross-Connection Scenarios

PII redaction (14.3), memory deletion (11.4), CI/CD controls (15), stakeholder communication (16.2).

Production Anti-Patterns

- Indefinite log retention of raw prompts.
- Deletion requests handled in the database but not the vector index.
- No tested kill switch or incident contact list.

Minimal Working Code

```
RETENTION_DAYS = {"public": 3650, "internal": 365,
 "confidential": 90, "regulated": 30}

def expired(records: list[dict], now_ts: float) -> list[str]:
    out = []
    for r in records:
        limit = RETENTION_DAYS[r["class"]] * 86400
        if now_ts - r["created"] > limit:
            out.append(r["id"])
    return out

print(expired([{"id": "r1", "class": "regulated", "created&q
uot;: 0}], 40 * 86400))
```

```
def erase_user(user_id: str, stores: dict) -> dict:
    """stores maps name -> callable(user_id)->int rows deleted. Returns an audit rec
ord."""
    audit = {}
    for name, delete_fn in stores.items():
        try:
            audit[name] = {"deleted": delete_fn(user_id), "error": None}
        except Exception as e:
            audit[name] = {"deleted": 0, "error": str(e)}
    audit["complete"] = all(v["error"] is None for v in audit.values() if isinstance(
v, dict))
    return audit

print(erase_user("u1", {"logs": lambda u: 4, "vectors": lambda u: 12, "
;memory": lambda u: 1}))
```

14.8 Mapping the OWASP LLM Top 10 to Concrete Controls

Conceptual Deep Dive

Turn risk lists into a control matrix with owners and tests. Representative mapping (names follow the OWASP Top 10 for LLM Applications; verify against the current edition): **Prompt injection** → untrusted-content labelling, least-privilege tools, approvals, injection classifiers, red-team regression suite. **Sensitive information disclosure** → PII redaction, output scanners, ACL-filtered retrieval, no secrets in prompts. **Supply chain** → pinned/hashed dependencies, signed images, safetensors only, vetted MCP servers. **Data and model poisoning** → provenance of training/RAG data, ingestion review, anomaly detection on new documents. **Improper output handling** → escape/validate outputs before rendering, SQL, shell. **Excessive agency** → minimal tool set, scoped credentials, rate/amount caps, human approval. **System prompt leakage** → assume leakage; keep secrets out. **Vector and embedding weaknesses** → tenant isolation, ACLs inside the query, embedding access control. **Misinformation** → citations, abstention, verification, domain review. **Unbounded consumption** → rate limits, token caps, quotas, budgets, timeouts. For each risk record: control, owner, test, monitoring signal. Edge cases: controls that exist on paper but have no test, and risks that change as you add tools or modalities.

Cross-Connection Scenarios

Threat modelling (14.2), red-teaming (14.6), guardrails (14.4), agent security (11.6).

Production Anti-Patterns

- A risk register that no one reviews after launch.
- Controls with no automated test.
- Treating the Top 10 as a one-time checklist.

Minimal Working Code

```
RISK_CONTROLS = {
    "prompt_injection": {"controls": ["untrusted_labelling", "least_pr
ivilege", "approvals"], "test": "redteam_injection"},
    "sensitive_disclosure": {"controls": ["pii_redaction", "acl_retrie
val"], "test": "leakage_probe"},
    "excessive_agency": {"controls": ["minimal_tools", "amount_caps&qu
ot;], "test": "tool_abuse_suite"},
    "unbounded_consumption": {"controls": ["rate_limits", "token_caps&
quot;, "budgets"], "test": "load_abuse"},
}

def coverage(implemented: set[str], tests_passing: set[str]) -> dict:
    report = {}
    for risk, spec in RISK_CONTROLS.items():
        missing = [c for c in spec["controls"] if c not in implemented]
        report[risk] = {"missing_controls": missing, "test_passing": spec["test&
quot;] in tests_passing}
    return report

print(coverage({"pii_redaction", "rate_limits", "token_caps", "budgets
"}, {"load_abuse"}))
```

```
def release_blockers(report: dict) -> list[str]:
    return [r for r, v in report.items() if v["missing_controls"] or not v["test_passing&q
uot;]]

print(release_blockers(coverage({"pii_redaction", "acl_retrieval"}, {"leakage_pr
obe"})))
```

14.9 Audit Logging, Evidence and Explainability

Conceptual Deep Dive

Regulated and enterprise customers need to answer "who did what, when, with which model and data?". An **audit record** per AI interaction should capture: request id, user/tenant (hashed where needed), timestamp, model and version, prompt template version, retrieval document ids (not necessarily content), tool calls with arguments and results (redacted), guardrail decisions, output hash, and outcome/feedback. Store immutably (append-only, write-once storage), with retention and access controls separate from application logs, and tamper evidence (hash chaining). **Explainability** for LLM systems is mostly *traceability*: show sources, retrieval scores, rules triggered and approvals, not claims about neural internals. Provide user-facing "why this answer" views and exportable reports for compliance. Edge cases: balancing audit completeness against privacy (log references instead of raw sensitive text), clock sync, and log volume costs.

Cross-Connection Scenarios

Tracing (14.1), PII (14.3), governance and retention (14.7), approvals (7.7).

Production Anti-Patterns

- Audit logs writable by the same service they audit.
- Storing raw prompts with PII indefinitely for "debugging".
- No link between a user complaint and the exact trace that produced the answer.

Minimal Working Code

```
import hashlib
import json
import time

class AuditLog:
    """Append-only, hash-chained log: altering any entry breaks every later hash."&qu
ot;"
    def __init__(self):
        self.entries, self.prev = [], "0" * 64

    def append(self, event: dict) -> dict:
        body = {"ts": time.time(), "prev": self.prev, **event}
        digest = hashlib.sha256(json.dumps(body, sort_keys=True).encode()).hexdigest()
        entry = {**body, "hash": digest}
        self.entries.append(entry)
        self.prev = digest
        return entry

    def verify(self) -> bool:
        prev = "0" * 64
        for e in self.entries:
            body = {k: v for k, v in e.items() if k != "hash"}
            if body["prev"] != prev or hashlib.sha256(json.dumps(body, sort_keys=True).encode()
).hexdigest() != e["hash"]:
                return False
            prev = e["hash"]
        return True

log = AuditLog()
log.append({"request_id": "r1", "model": "m-1", "prompt_vers
ion": "a1b2", "docs": ["d4", "d9"], "action": &quo
t;answered"})
log.append({"request_id": "r2", "tool": "refund", "approved_
by": "agent_17"})
print(log.verify())
log.entries[0]["model"] = "tampered"
print(log.verify())
```

```
def audit_record(request_id: str, user_id: str, output: str, **meta) -
> dict:
    return {"request_id": request_id,
            "user": hashlib.sha256(user_id.encode()).hexdigest()[:16],
```

```
        "output_hash": hashlib.sha256(output.encode()).hexdigest(),
        **meta}

print(audit_record("r9", "alice@example.com", "Your refund was approved.",
model="m-1", docs=["d2"]))
```

14.10 Copyright, Provenance, Watermarking and Regulation

Conceptual Deep Dive

Legal and policy landscapes shift quickly; treat this as an engineering checklist and consult counsel. **Training and RAG data**: track licences and sources; honour robots/opt-outs where required; avoid ingesting content you cannot lawfully store or quote; keep deletion capability. **Outputs**: models can reproduce memorised text or code; mitigate with duplicate-detection filters, licence scanners for generated code, and quoting limits. **Provenance**: attach metadata to generated media (C2PA content credentials) and keep internal records of what was AI-generated. **Watermarking** (statistical token-level or image/audio watermarks) helps detection but is imperfect and removable. **Disclosure**: tell users when they interact with AI and when content is AI-generated where law or policy requires. **Regulation** to know at a high level: the EU AI Act (risk tiers, transparency duties, general-purpose model obligations), GDPR/UK GDPR (lawful basis, DPIAs, data-subject rights), CCPA/CPRA, sectoral rules (HIPAA, financial regulators), and standards (NIST AI RMF, ISO/IEC 42001). Edge cases: cross-border data flows, models trained on data that later receives a deletion request, and jurisdiction-specific restrictions on automated decisions.

Cross-Connection Scenarios

Responsible AI (1.8), governance (14.7), synthetic data licensing (12.10), model cards (16.6).

Production Anti-Patterns

- Shipping generated code with no licence scan in a commercial product.
- No record of data sources for a fine-tuned model.
- Marketing claims that exceed what the system can guarantee.

Minimal Working Code

```
ALLOWED_LICENSES = {"mit", "apache-2.0", "bsd
-3-clause", "cc-by-4.0", "cc0-1.0"}

def audit_sources(sources: list[dict]) -> dict:
    blocked = [s["name"] for s in sources if s.get("license", "").lower() n
ot in ALLOWED_LICENSES]
    return {"ok": not blocked, "blocked": blocked}

print(audit_sources([{"name": "wiki-dump", "license": "CC-BY-4.0"
}, {"name": "scraped-forum", "license": ""}]))
```

```
def verbatim_overlap(output: str, corpus_ngrams: set[tuple], n: int =
8) -> float:
    toks = output.lower().split()
    grams = [tuple(toks[i:i + n]) for i in range(len(toks) - n + 1)]
    if not grams:
        return 0.0
    return sum(g in corpus_ngrams for g in grams) / len(grams)

ref = "to be or not to be that is the question whether tis nobler".split()
corpus = {tuple(ref[i:i + 8]) for i in range(len(ref) - 7)}
print(verbatim_overlap("to be or not to be that is the question", corpus))
```